UNIVERSIDAD EAFIT · Escuela de Ciencias Aplicadas e Ingeniería
Departamento de Informática y Analítica

Seguridad de Archivos mediante Intervención en la Capa del Kernel y Mapeo de Memoria (mmap)

Evaluación parcial práctico-teórica (25%) · Sistemas Operativos · Periodo 2026-2

Solución implementada: **MMapGuard**

Autores
Miguel Montoya · Samuel Alarcón · Manuel Arenas

Repositorio: https://github.com/ManuelArenas1/mmapguard
Video explicativo: `video/mmapguard_demo.mp4` (en el repositorio)

Contenido y resumen ejecutivo

Este informe reúne las tres partes que pide el parcial:

Parte	Peso	Sección
I — Análisis conceptual: 10 preguntas diagnósticas	30%	Sección 1
II — Estrategia con IA: flujo de trabajo, 5 prompts, alucinaciones, arquitectura y *trade-offs*	30%	Sección 2
III — Implementación, argumentación, overhead y limitaciones	40%	Sección 3

Solución en una frase. `mmapguard` es un módulo de kernel (LKM) que engancha con *kretprobes* las funciones del framework de seguridad `security_mmap_file()` y `security_file_mprotect()` para auditar o denegar (`-EPERM`) el mapeo de archivos protegidos, identificados por `(dispositivo, inodo)`. Se controla por `/proc/mmapguard` (modos `off|audit|enforce`, `strict`).

Estado de verificación (declarado con honestidad). El módulo compila sin advertencias (`-Wall -Werror`) contra los headers de Linux 6.8. Las cuatro PoC de usuario se ejecutaron como línea base sin el módulo. La carga del módulo (`insmod`) no pudo hacerse en el entorno de desarrollo, que es un contenedor sin soporte de módulos; el script `scripts/run_tests.sh` automatiza la demostración completa para una VM Linux real.

Contenido del repositorio: `kmod/` (módulo), `tests/` (4 PoC y benchmark), `scripts/run_tests.sh`, `docs/` y `video/`.

Parte I — Análisis conceptual y preguntas diagnósticas (30%)

> Cada respuesta se fundamenta a nivel de arquitectura del kernel de Linux (estructuras internas, MMU, flujo de eventos) y, cuando aplica, se referencia directamente al código implementado en `kmod/mmapguard.c` y a las pruebas de concepto en `tests/`.

---

1. Mapeo virtual y Page Faults: secuencia exacta `mmap()` → primer Page Fault

1. **Espacio de usuario**: el proceso invoca `mmap(addr, len, prot, flags, fd, offset)`. En glibc esto es una envoltura de la syscall `sys_mmap` / `__x64_sys_mmap`.
2. **Entrada a Ring 0**: la CPU ejecuta la instrucción `syscall`, guarda el contexto de usuario y transfiere el control a `entry_SYSCALL_64` → `do_syscall_64` → `sys_mmap` → `ksys_mmap_pgoff()` → `vm_mmap_pgoff()` → `do_mmap()`.
3. **Validaciones y creación del VMA**: `do_mmap()` valida los argumentos (alineación, límites de `mmap_min_addr`, `RLIMIT_AS`), y **antes de tocar ninguna tabla de páginas**, invoca el framework de seguridad: `security_mmap_file(file, prot, flags)`. Este es exactamente el punto de intercepción que usa `mmapguard` (ver pregunta 4). Si el chequeo retorna `0`, se continúa.
4. **Reserva del rango virtual**: el kernel busca un hueco libre en `mm->mmap` (el árbol de VMAs, en kernels modernos un *maple tree*, `mm->mm_mt`) mediante `get_unmapped_area()`, y construye un nuevo `struct vm_area_struct` que describe `[vm_start, vm_end)`, las banderas (`vm_flags`: `VM_READ`, `VM_WRITE`, `VM_EXEC`, `VM_SHARED`…), y el `struct file *vm_file` + `vm_pgoff` si el mapeo está respaldado por archivo.
5. **Instalación en la tabla de páginas (sin contenido aún)**: `do_mmap()` **no** copia datos ni crea las entradas finales de la tabla de páginas (PTE). Solo registra el VMA. Las entradas PTE del rango quedan ausentes (`not present`) — este es el corazón de la paginación por demanda.
6. **Retorno a Ring 3**: la syscall retorna la dirección virtual `addr` al proceso. Hasta este momento, el kernel ha gastado CPU únicamente en contabilidad de VMAs; no se ha tocado una sola página física de datos del archivo.
7. **Primer acceso del proceso**: cuando el proceso ejecuta una instrucción `MOV` (lectura o escritura) sobre una dirección dentro del rango mapeado, la **MMU** traduce la dirección virtual recorriendo las tablas de páginas (`PGD → P4D → PUD → PMD → PTE` en x86_64 de 4/5 niveles). Como la entrada PTE está marcada *not present*, la MMU genera la excepción de hardware **`#PF` (vector 14, Page Fault)**.
8. **Manejo del Page Fault en el kernel**: la CPU transfiere el control a `asm_exc_page_fault` → `exc_page_fault()` → `do_user_addr_fault()` → `handle_mm_fault()` → `__handle_mm_fault()` → `handle_pte_fault()`. El kernel localiza el VMA correspondiente (`find_vma()`), confirma que la dirección cae dentro de un VMA válido y que el tipo de acceso es compatible con `vma->vm_flags` (si no, el proceso recibe `SIGSEGV`).
9. **Resolución del fault para un mapeo de archivo**: como el VMA tiene `vm_file`, el kernel delega en `vma->vm_ops->fault` (para archivos regulares, normalmente `filemap_fault()` del *page cache*). Este busca la página en el *Page Cache* (`address_space->i_pages`, un *xarray*); si no está en caché, dispara I/O de bloque (`->read_folio`) para traerla desde disco, y la deja residente en RAM.
10. **Instalación final de la PTE**: con la página física ya disponible (`struct page` / `struct folio`), el kernel escribe la entrada PTE apuntando al *Page Frame Number* (PFN) correspondiente, con los bits de protección (`R/W`, `U/S`, `NX`) derivados de `vm_page_prot`. La instrucción que falló se

reejecuta — esta vez la MMU encuentra la PTE presente y el acceso se resuelve **a velocidad de hardware**, sin volver a entrar a Ring 0.

Conclusión arquitectónica: `mmap()` es "barato" en la syscall porque solo crea metadatos (`vm_area_struct`); el costo real de traer datos se paga de forma diferida y granular (4 KB a la vez) en cada Page Fault — este es precisamente el único momento, junto con la syscall inicial, en que el kernel puede tomar decisiones de seguridad sobre el contenido. `mmapguard` interviene en el paso 3 (creación del VMA) porque es la última oportunidad confiable antes de que el acceso se vuelva invisible a nivel de syscall.

---

2. `MAP_SHARED` vs `MAP_PRIVATE`: gestión en `vm_area_struct` e implicaciones de seguridad

El `struct vm_area_struct` no almacena un booleano "shared" explícito como tal; la semántica se codifica en `vma->vm_flags` mediante el bit `VM_SHARED`, derivado del parámetro `flags` de `mmap()` en el momento de `do_mmap()`:

```
        if ((flags & MAP_TYPE) == MAP_SHARED) vm_flags |= VM_SHARED | VM_MAYSHARE;
```

- `MAP_SHARED`: todas las referencias al mismo rango de archivo (mismo `(dev, ino, offset)`) comparten las **mismas páginas físicas** a través del *Page Cache* del `address_space` del inodo. Dos procesos distintos que mapeen el mismo archivo con `MAP_SHARED` obtienen VMAs con direcciones virtuales distintas, pero que apuntan al **mismo PFN**. Las escrituras son visibles instantáneamente entre procesos (sin sincronía explícita) y, eventualmente, se reflejan en el archivo en disco (`writeback`, disparado por `msync()`, `fsync()` o el *pdflush*/`kworker` de escritura diferida).
  - **Implicación de seguridad**: una escritura con `PROT_WRITE|MAP_SHARED` modifica el archivo real sin pasar jamás por `write()` / `copy_from_user()` — es indetectable con herramientas basadas en syscalls (`strace`, `LD_PRELOAD`) una vez que el VMA existe. Esta es exactamente la amenaza que `mmapguard` neutraliza (PoC `poc_shared_write.c`).
- `MAP_PRIVATE`: el VMA se marca con `VM_MAYWRITE` pero **no** `VM_SHARED`. Las páginas se mapean inicialmente como **solo lectura** compartidas con el *Page Cache* (igual que `MAP_SHARED` para la lectura), pero cualquier escritura dispara **Copy-On-Write (COW)**: el kernel asigna una página física *anónima* nueva, copia el contenido, y re-mapea la PTE del proceso hacia esa copia privada, marcándola `dirty` y quitándole la bandera de solo-lectura que forzaba el COW.
  - **Implicación de seguridad**: el archivo en disco **nunca** se modifica por esta vía (la copia es exclusiva del proceso y se descarta al desmapear), pero el proceso sí puede alterar su **propia vista** de los datos — relevante para integridad de datos en memoria del proceso, no del archivo. Es la base conceptual, mal entendida, detrás de ataques como Dirty COW (pregunta 9).

En `mmapguard`, la clasificación "shared vs private" se obtiene leyendo `vma->vm_flags & VM_SHARED` en el hook de `mprotect` y el bit `MAP_TYPE` de los flags crudos en el hook de `mmap` (ver `mg_evaluate()` en `kmod/mmapguard.c`), que es exactamente el mecanismo que usa el propio kernel.

---

3. Mecanismo Copy-On-Write (COW): ciclo de vida y riesgos de escalamiento

Ciclo de vida de una página bajo COW (mapeo `MAP_PRIVATE`, `PROT_READ|PROT_WRITE`, archivo abierto en modo lectura o lectura/escritura):

1. **Mapeo inicial**: el VMA se crea con `vm_flags = VM_READ|VM_WRITE|VM_MAYWRITE` (el proceso *puede* escribir eventualmente), pero la PTE real que instala el primer Page Fault de lectura es **de solo lectura a nivel de hardware** (`vma->vm_page_prot` fuerza `_PAGE_RW=0`), aunque `vm_flags` diga `VM_WRITE`. Esta discrepancia intencional entre "permiso lógico" (`vm_flags`) y "permiso físico en la PTE" es el mecanismo que permite que el kernel intercepte la *primera* escritura.
2. **Primera escritura → Page Fault de protección**: cuando el proceso escribe, la MMU detecta que la PTE es de solo lectura y genera `#PF` con el bit de error `WRITE` activo pero la página sí "presente" → esto es un **fault de protección**, no de ausencia, y cae en `handle_pte_fault()` → `do_wp_page()` (*write-protect page*).
3. **Decisión de COW**: `do_wp_page()` verifica el contador de referencias de la página (`page_count`/`folio_ref_count` vía `PageAnonExclusive`/mapcount). Si la página está compartida con el *Page Cache* o con otro proceso, se asigna una página anónima nueva (`alloc_page_vma`), se copia el contenido byte a byte, se actualiza la PTE del proceso para apuntar a la copia con permisos R/W reales, y se decrementa la referencia a la página original.
4. **Página "rota" (unshared)**: a partir de este punto la página es exclusiva del proceso, vive en la lista de páginas anónimas de la `mm_struct` (contabilizada como *swap-backed*), y las escrituras subsecuentes son directas, sin más faults de protección.

Condiciones de fallo que derivan en escalamiento/corrupción: el riesgo no está en el COW en sí (que es correcto y está bien probado), sino en **rutas alternativas que modifican la página *antes* de que el COW se complete o la evaden**, en particular:

- Si el kernel permite que un *Get User Pages* con `FOLL_WRITE` forzado (`FOLL_FORCE`, usado por `/proc/pid/mem`, `ptrace`, `process_vm_writev`) escriba sobre la página **compartida original** antes de que `do_wp_page()` la "rompa" (la ruta de GUP tenía su propia lógica de COW, separada de la del fault handler) — esto es exactamente la causa raíz de **Dirty COW / CVE-2016-5195** (ver pregunta 9).
- Si `VM_MAYWRITE` queda activo en un VMA que *no debería* permitir escritura futura (p. ej., un binario ejecutable mapeado `MAP_PRIVATE` de un `fd` de solo lectura), y una ruta de kernel bugueada confía en `vm_flags` sin revalidar `PROT_WRITE`/`mprotect` en cada operación sensible.
- Combinaciones con `PROT_EXEC`: una página rota por COW que **conserva** el bit `NX=0` (ejecutable) puede convertirse en un vector de **inyección de código** si el atacante logra escribir contenido arbitrario en ella antes de que se marque de solo-ejecución — de ahí la política *strict* de `mmapguard`, que también niega `PROT_EXEC` sobre archivos protegidos.

---

4. Intercepción en el kernel: LKM con hook en la *System Call Table* vs. LSM (`file_mmap`)

Criterio	Hook en `sys_call_table` (LKM clásico)	LSM (`security_mmap_file`, hooks `file_mmap`)
Viabilidad actual (kernel ≥ 5.7)	**Prácticamente inviable**: `sys_call_table` ya no se exporta (`EXPORT_SYMBOL`) desde el kernel 5.7+ (hay que resolverla por *kallsyms*/escaneo de memoria, frágil entre versiones). Además, desde el 3.3/CONFIG_ARCH_HAS_SET_MEMORY, la tabla y el segmento de texto del kernel suelen estar mapeados **solo lectura** (`CR0.WP` *pinned*, `set_memory_ro`), por lo que escribirla requiere deshabilitar protecciones de página a mano (`write_cr0`), algo que **rompe SMEP/KPTI** y es tratado como *rootkit behavior* por EDRs y por el propio kernel con `CONFIG_STRICT_KERNEL_RWX` .	**Viable y soportado**: `security_mmap_file()` es una función no estática, exportada, llamada explícitamente por `do_mmap()` en cada `mmap()` . No requiere parchear memoria protegida.
Estabilidad entre versiones	Baja: el *layout* exacto de la tabla, el *calling convention* (`asmlinkage`), y el número de syscall pueden cambiar.	Alta: la firma de `security_mmap_file(struct file*, unsigned long, unsigned long)` es estable desde hace muchas versiones LTS; es parte de una API pública del subsistema de seguridad.
Cobertura semántica	Solo ve `sys_mmap` tal cual se invocó; no ve mmaps hechos internamente por el kernel en nombre de un proceso (ej. carga de binarios vía `execve`), ni es trivial correlacionar con el VMA resultante.	Se ejecuta con el `struct file*` ya resuelto y **antes** de que el VMA exista; es el mismo punto donde SELinux/AppArmor/Smack toman decisiones reales — máxima fidelidad semántica.
Permite modificar/denegar	Sí, en teoría (reemplazando el puntero de función), pero de forma insegura y no soportada.	Sí: devolver `!= 0` desde el hook hace que `do_mmap()` retorne ese error sin crear el VMA. Es el mecanismo **diseñado** para esto.
Limitación real para un LKM (no para un LSM in-tree)	—	Un **módulo cargable NO puede registrarse como LSM** en kernels modernos: la lista de *security modules* habilitados se fija en tiempo de compilación/arranque (`CONFIG_LSM=...` , parámetro de arranque `lsm=`) via `security_add_hooks()` llamado solo desde código *built-in* marcado `__initdata` . Esto es intencional: evita que un rootkit se "inserte" como LSM dinámicamente.

Conclusión y decisión de diseño: ni "hookear la syscall table" ni "escribir un LSM real" son viables/seguros para un LKM externo en un kernel moderno. La alternativa correcta —y la que usa `mmapguard` — es un **kretprobe** sobre la función del framework LSM (`security_mmap_file` , `security_file_mprotect`). El *kprobe* no parchea memoria permanentemente: usa el framework oficial de *tracing* del kernel (`ftrace` /*breakpoints* dinámicos), que es la vía soportada para instrumentar funciones del kernel en caliente, y permite inspeccionar los argumentos en la entrada y **sustituir el valor de retorno** en el `return-handler` , logrando el mismo efecto que tendría un LSM real (el `do_mmap()` ve un `security_mmap_file() != 0` y aborta con `-EPERM`) sin necesidad de recompilar el kernel ni de registrar un LSM.

---

5. Cifrado transparente en memoria para archivos confidenciales mapeados

Para que un archivo cifrado en disco sea legible en claro vía `mmap()` **solo** para procesos autorizados, sin exponer el texto en claro en memoria física a otros procesos no privilegiados, la pila de componentes debe cooperar así:

1. **Descifrado en la frontera disco↔Page Cache, no en la frontera Page Cache↔proceso**: el *Page Cache* almacena **páginas en claro** (es la única forma de que `mmap()` funcione de forma transparente, porque el *Page Cache* es lo que se mapea directamente). El cifrado debe residir en el **driver de bloque o el filesystem** (ej. `fscrypt` para cifrado por-archivo en ext4/F2FS, o `dm-crypt` por debajo del bloque completo), de forma que el descifrado ocurra en el camino `->read_folio()` (al traer la página desde el dispositivo hacia el Page Cache) y el cifrado ocurra en `->writepage()` / `writeback` (al bajar una página sucia de vuelta a disco). El contenido que vive en RAM dentro del Page Cache **ya está en claro**.
2. **El problema real no es "cómo descifrar", sino "cómo evitar que ese Page Cache en claro sea visible a un proceso no autorizado"**: como `MAP_SHARED` comparte el **mismo** `struct page` entre todos los procesos que abren el archivo, **no existe una forma nativa de dar una vista en claro a un proceso y una vista cifrada (o nada) a otro sobre las mismas páginas físicas** — el Page Cache es por-inodo, no por-proceso.
3. **Mecanismos reales para lograr aislamiento**:
   - **Control de acceso en la apertura/mapeo**: negar el `open()` (permisos POSIX/ACL/SELinux) o negar el propio `mmap()` (exactamente el rol de `security_mmap_file` / `mmapguard`) a procesos no autorizados, de forma que únicamente el proceso correcto llegue a tener un VMA sobre ese inodo.
   - **Filesystems cifrados por clave-de-proceso** (ej. `fscrypt` con claves por *keyring* de sesión): el kernel mantiene *page cache* separado cuando las claves de descifrado difieren, de modo que un proceso sin la clave correcta ni siquiera puede completar el `open()` con éxito (obtiene `ENOKEY`), evitando que llegue a existir una página en claro accesible para él.
   - **Enclaves de hardware (SGX/TDX/SEV-SNP)**: el verdadero aislamiento *memoria-a-memoria* frente a otros procesos con el mismo UID/root requiere soporte de hardware que cifre la RAM física fuera del enclave — el kernel por sí solo, con una CPU convencional, no puede impedir que un proceso con acceso directo a `/dev/mem` o con `CAP_SYS_RAWIO` / `ptrace` sobre el proceso autorizado lea esas páginas.
4. **Rol de `mmapguard` en esta cadena**: no implementa cifrado (eso es responsabilidad del filesystem/driver), pero sí es el **punto de control de acceso** descrito en el punto 3: decide *quién* llega a crear el VMA sobre el archivo sensible, lo cual es condición necesaria (aunque no suficiente sin cifrado en disco) para la confidencialidad.

---

6. `msync` / `fsync` y consistencia de páginas sucias; evasión si no se inspecciona en el momento de escritura

Mecanismo de consistencia: cuando un proceso escribe sobre una página `MAP_SHARED` ya con PTE presente, la **MMU** simplemente setea el bit hardware `DIRTY` en la PTE — esto ocurre sin ninguna intervención del kernel (es automático del hardware x86/ARM). El kernel se entera de que una página está sucia de dos formas:

- **De forma perezosa/periódica**: los *kworkers* de *writeback* (`wb_workfn()`) recorren inodos con páginas sucias (`mapping->i_pages` + `PAGECACHE_TAG_DIRTY`) y las bajan a disco cada cierto intervalo (`dirty_writeback_centisecs`, por defecto 5 s) o cuando se supera un umbral de memoria sucia (`dirty_ratio` / `dirty_background_ratio`).
- **De forma explícita y sincrónica**: `msync(addr, len, MS_SYNC)` (o su análogo a nivel de fd, `fsync()`) fuerza a `filemap_write_and_wait_range()` a recorrer exactamente ese rango, escribir las páginas sucias al dispositivo y esperar la confirmación de I/O antes de retornar.

El hueco de inspección: dado que el hardware marca `DIRTY` sin pasar por ninguna función del kernel, **no existe un "hook" nativo que se dispare en cada byte escrito** dentro de una página ya mapeada. Un filtro de seguridad que solo observe `write()` / `pwrite()` (I/O clásica) **nunca verá** estas escrituras. Un atacante que logró obtener un VMA `MAP_SHARED|PROT_WRITE` sobre un archivo puede escribir contenido arbitrario y, si ningún control inspecciona el *contenido* en el momento del `msync` /*writeback* (solo se verificó el **permiso de abrir el mapeo**, no cada escritura posterior), el dato llega a disco sin inspección alguna — exactamente lo que demuestra `poc_shared_write.c`.

Mitigaciones reales, en orden de intrusividad creciente:

1. **Negar la creación del VMA de escritura** (enfoque de `mmapguard`): más simple y de costo casi nulo, pero es "todo o nada" — no permite escritura parcial inspeccionada.
2. **`vm_operations_struct->page_mkwrite`**: el kernel sí expone un hook que se dispara en la **primera** transición de una página `MAP_SHARED` de limpia a sucia (usado por filesystems para reservar bloques en disco antes de permitir la escritura, *delayed allocation*). Un driver propio puede usar este hook para auditar/vetar la primera escritura de cada página, pero no ve byte-a-byte el contenido subsecuente dentro de esa misma página ya "abierta" para escritura.
3. **`madvise(MADV_PAGEOUT)` /revocar la PTE tras cada escritura**: forzar artificialmente que cada acceso vuelva a generar un fault (quitando el bit *present* después de cada `page_mkwrite`) permite inspección por página a costa de un overhead altísimo (vuelve cada escritura tan cara como un `write()`, anulando la ventaja de `mmap()`).

---

7. Auditoría con eBPF (kprobes/fexit) sin recompilar el kernel

eBPF permite cargar programas verificados en una VM dentro del kernel, adjuntos a puntos de instrumentación dinámicos, sin tocar el código fuente ni reiniciar:

- **`kprobe` / `kretprobe` genéricos**: se adjuntan a *cualquier* símbolo del kernel exportado en *kallsyms* (ej. `security_mmap_file`, `do_madvise`, `handle_mm_fault`). El programa eBPF corre en la entrada (o salida, para `kretprobe`) de la función, puede leer argumentos (vía `BPF_CORE_READ` para resistir cambios de *offsets* entre versiones — *CO-RE*), y escribir eventos a un `BPF_MAP_TYPE_RINGBUF` que un proceso de usuario (`libbpf`) consume en tiempo real.
- **`fentry` / `fexit`** (BPF trampolines): mecanismo más moderno y de **menor overhead** que los kprobes clásicos (evita el *breakpoint trap* de `int3` / `ftrace`, usando en su lugar un trampolín generado por JIT que llama directamente al programa BPF). `fexit` es particularmente útil porque entrega tanto los argumentos **como el valor de retorno real** de la función, permitiendo decidir auditoría *después* de conocer si el kernel permitió o no la operación.
- **`bpf_lsm` (LSM basado en eBPF)**: desde el kernel 5.7, es posible adjuntar programas eBPF directamente a los *hooks* LSM (incluyendo `mmap_file`) usando `BPF_PROG_TYPE_LSM`, y estos **sí pueden alterar el resultado** (retornar `-EPERM`) de forma soportada — a diferencia de un kprobe convencional, esto es una ruta oficialmente diseñada para ese propósito, aunque requiere `CONFIG_BPF_LSM=y` y que el LSM `bpf` esté habilitado en la lista de LSMs activos del

sistema (parámetro `lsm=` en el *cmdline* del kernel), algo que no siempre está disponible fuera de distros recientes configuradas para ello.
- **Patrón típico de auditoría de accesos anómalos sobre `mmap()`**: adjuntar `kprobe`/`fentry` a `security_mmap_file` y `security_file_mprotect` (lectura de argumentos), correlacionar en un mapa eBPF (`BPF_MAP_TYPE_HASH` por `pid`/`inode`) con eventos de `do_madvise` (detectar `MADV_DONTNEED`), y exportar al *ring buffer* cualquier secuencia temporal sospechosa (p. ej., `MADV_DONTNEED` seguido en pocos microsegundos de una escritura vía `/proc/pid/mem`) — exactamente el patrón que la PoC `poc_race_madvise.c` reproduce de forma controlada, y que en este proyecto se detecta/comenta por la vía de kprobes/kretprobes del propio LKM en lugar de eBPF (ver sección de limitaciones en la Parte III sobre por qué se eligió kprobe-LKM sobre eBPF para este entregable).

---

8. Evasión de permisos POSIX: `O_RDONLY` + `mmap(PROT_WRITE, MAP_PRIVATE)`

Sí, el kernel lo permite, y es un comportamiento documentado, no un bug: `mmap(2)` exige que los permisos de apertura del descriptor sean **compatibles como mínimo** con `PROT_READ` para cualquier mapeo, y con `PROT_WRITE` **solo si** `flags` incluye `MAP_SHARED` (en cuyo caso el kernel exige que el `fd` haya sido abierto con `O_RDWR`, verificado en `do_mmap_pgoff()` / `vma_wants_writenotify()`). Pero si `flags = MAP_PRIVATE`, el kernel permite `PROT_WRITE` incluso con un `fd` abierto `O_RDONLY`, **porque la semántica de `MAP_PRIVATE` garantiza que la escritura jamás se propaga al archivo**: cualquier escritura primero pasa por COW (pregunta 3), produciendo una página anónima privada; el archivo subyacente, al que el proceso solo tiene permiso de lectura, nunca se toca.

El propio código del kernel lo valida explícitamente en `do_mmap()`:

```
        if (flags & MAP_SHARED) {
            if (prot & PROT_WRITE) {
                if (!(file->f_mode & FMODE_WRITE))
                    return -EACCES;    /* MAP_SHARED exige fd con permiso real de escritura */
            }
        }
        /* MAP_PRIVATE con PROT_WRITE sobre fd O_RDONLY: no hay chequeo adicional,
         * porque COW garantiza que nunca se llega a escribir el inodo. */
```

Riesgos que esto sí introduce (demostrados en `tests/poc_private_cow.c`):

1. **Alteración temporal de datos "en caliente"**: el proceso puede modificar su copia en memoria de un archivo que el sistema de permisos le dijo que era "solo lectura" para él. Si ese proceso luego **comparte esa memoria** con otro componente (ej. la pasa a un hijo con `fork()`, o la expone via otro `mmap(MAP_SHARED, /* de memoria anónima compartida */)`), puede inducir a otros componentes a confiar en datos alterados sin que el archivo real haya cambiado — un vector de **confusión de integridad**, no de fuga.
2. **Fuga de información indirecta**: no hay fuga de contenido *adicional* al que ya tenía por `PROT_READ` (el proceso ya podía leer todo el archivo); el riesgo real es que un atacante use este patrón como **paso 1** de un ataque de carrera tipo Dirty COW (pregunta 9), combinando esta "escritura permitida sobre la copia" con una forma de redirigir esa escritura hacia la página original *antes* de que el COW termine de separarla.
3. **Mitigación de `mmapguard`**: el modo **strict** niega explícitamente `MAP_PRIVATE + PROT_WRITE` sobre archivos protegidos, precisamente porque aunque el archivo esté seguro en sí mismo, este patrón es el primer paso observable de las PoC de carrera.

---

9. Mitigación de ataques de carrera: `madvise(MADV_DONTNEED)` vs escrituras sobre páginas privadas (Dirty COW)

El ataque histórico (CVE-2016-5195, "Dirty COW", 2007–2016): la ruta de *Get User Pages* (`get_user_pages()`, usada por `/proc/pid/mem`, `ptrace(POKETEXT)`, `process_vm_writev`) para escribir con `FOLL_WRITE|FOLL_FORCE` sobre una página de un mapeo `MAP_PRIVATE` de solo lectura (`vma->vm_flags` sin `VM_WRITE` real a nivel PTE) tenía su **propia** lógica de COW, separada de la de `do_wp_page()` (el *fault handler* normal). La carrera:

1. **Hilo A** (`madvise(MADV_DONTNEED)` en bucle): descarta la página privada ya "rota" por COW que el proceso tenía, haciendo que la próxima referencia vuelva a mapearse directamente a la página **original del Page Cache** (de solo lectura, compartida con el archivo en disco).
2. **Hilo B** (escritura vía `/proc/self/mem` con `pwrite`): en la estrecha ventana de tiempo en que la PTE vuelve a apuntar al Page Cache original pero **antes** de que el camino de GUP complete su propio COW, la escritura de `FOLL_FORCE` se aplicaba directamente sobre la página **compartida real**, modificando permanentemente el archivo en disco (incluso uno de solo lectura para el usuario), porque el camino de escritura forzada de GUP no revalidaba atómicamente el estado COW frente al `madvise` concurrente.

Cómo lo previene el kernel parcheado (desde 4.8.3/4.7.9, commit `19be0eaffa3ac7d8eb6784ad9bdbc7d67ed8e619`, "mm: remove gup_flags FOLL_WRITE games from __get_user_pages()"): se introdujo la bandera `FOLL_COW`, que obliga a la ruta de GUP a **revalidar explícitamente**, bajo el mismo `page table lock` / `mmap_lock` que usa `do_wp_page()`, que la página a escribir ya pasó por COW y es verdaderamente privada/exclusiva del proceso (`can_follow_write_pte()` verifica `PageAnonExclusive`), en lugar de confiar en el estado de la PTE leído momentos antes. Esto cierra la ventana de carrera: cualquier intento de escribir vía GUP forzado sobre una página que *todavía* es la compartida del Page Cache obliga, en la misma operación atómica, a ejecutar el COW real antes de escribir — nunca se escribe la página original.

Rol de `mmapguard` frente a esto: dado que el *fix* ya está en el kernel mainline desde 2016, `tests/poc_race_madvise.c` reproduce fielmente el **patrón** de la carrera (sin ninguna vulnerabilidad real) y, al ejecutarse sobre un kernel moderno, **siempre** reporta `[KERNEL PARCHEADO]` — es decir, verifica que el entorno de ejecución tiene el *fix* de `FOLL_COW` activo. `mmapguard` en modo *strict* añade una capa de **defensa en profundidad**: como niega de entrada `MAP_PRIVATE+PROT_WRITE` sobre archivos protegidos, ni siquiera se llega a crear el VMA que un futuro bug similar podría explotar — es decir, reduce la superficie de ataque independientemente de si existe o no una vulnerabilidad de carrera vigente.

---

10. `struct page`, tablas de páginas (PGD/P4D/PUD/PMD/PTE) y bits NX/R-W/U-S en la contención de inyección de código

- **`struct page`** (o `struct folio` en kernels recientes ≥ 5.17, que agrupa páginas compuestas): es el descriptor **por página física** que el kernel usa para rastrear su estado — contador de referencias (`_refcount`), si está mapeada y por cuántas PTEs (`_mapcount`), si pertenece al Page Cache de un inodo o es anónima, si está sucia, bloqueada, reservada, etc. Es la unidad de contabilidad que permite al kernel saber, en cualquier momento, **quién** tiene acceso a una página física dada — condición necesaria para poder razonar sobre aislamiento.
- **Jerarquía de tablas de páginas (x86_64, 4 niveles: PGD→PUD→PMD→PTE, o 5 con P4D en sistemas con más de 48 bits de dirección)**: cada proceso tiene su propio **PGD**

(apuntado por el registro `CR3`, cambiado en cada *context switch*), de forma que el mismo rango de direcciones virtuales en dos procesos distintos puede traducir a páginas físicas completamente distintas (o a la misma, si comparten un mapeo). Esta jerarquía es la base del **aislamiento de espacios de direcciones**: un proceso no puede, por diseño de hardware, generar una dirección virtual que resuelva a una página física para la cual no existe una PTE válida en *su propio* árbol de tablas.

- **Bits de protección en la PTE (leídos por la MMU en cada acceso, no por software)**:
  - **R/W** (bit de escritura): si está en `0` (solo lectura) y la CPU intenta escribir, se genera `#PF` de protección — es el mecanismo que habilita COW (pregunta 3) y `page_mkwrite` (pregunta 6).
  - **U/S** (*User/Supervisor*): separa qué páginas son accesibles desde Ring 3 vs. solo desde Ring 0. Combinado con **SMEP** (*Supervisor Mode Execution Prevention*) y **SMAP** (*Supervisor Mode Access Prevention*) a nivel de CPU, impide que, incluso si el kernel fuera engañado para saltar a una dirección controlada por el atacante en espacio de usuario, lo haga con privilegios de Ring 0, o que lea/escriba memoria de usuario sin usar explícitamente `copy_to_user` / `copy_from_user` (que activan temporalmente el permiso).
  - **NX/XD** (*No-eXecute* / *eXecute Disable*, bit 63 de la PTE): si está activo, la CPU genera `#PF` ante cualquier intento de **ejecutar** instrucciones desde esa página, independientemente de los permisos R/W. Es la defensa de hardware central contra inyección de código: una página de datos (heap, stack, una región anónima producto de COW) se mapea típicamente con `NX=1`, de modo que aunque un atacante logre escribir shellcode en ella (vía un buffer overflow, por ejemplo), **no puede hacer que la CPU lo ejecute** directamente ahí — debe recurrir a técnicas más complejas (ROP/JOP reutilizando código ya ejecutable existente, *W^X bypass*).
- **Contención de inyección en el contexto de este proyecto**: una región mapeada con `mmap()` que combine `PROT_WRITE` y `PROT_EXEC` simultáneamente (`W^X` violado) es el escenario de mayor riesgo, porque permite escribir datos arbitrarios y luego ejecutarlos en la misma página sin pasar por `NX`. Por esto, el modo *strict* de `mmapguard` trata `PROT_EXEC` sobre archivos protegidos como motivo de denegación igual que las escrituras: el objetivo es nunca permitir que una página respaldada por un archivo sensible llegue a tener, simultáneamente o en secuencia vía `mprotect`, permisos de escritura y ejecución.

Parte II — Diseño de la estrategia con apoyo de Inteligencia Artificial (30%)

3.1 Definición del flujo de trabajo asistido por IA

El proyecto se desarrolló con **Claude** (Anthropic) como copiloto de ingeniería a lo largo de tres fases, cada una con un rol distinto y distinto nivel de confianza otorgado a la salida del modelo:

Fase	Rol de la IA	Nivel de confianza en la salida
Exploración arquitectónica	Explicar estructuras del kernel (`vm_area_struct`, `security_mmap_file`, `kretprobe`), comparar alternativas de intercepción (syscall table vs. LSM vs. kprobe vs. eBPF) y su viabilidad en kernels ≥ 5.7, y leer/sintetizar el repositorio base del curso (`SO2026B/CapaKernel`) para alinear el estilo y las convenciones del proyecto con el material visto en clase.	Alta para conceptos estables y bien documentados (ABI de syscalls, ciclo de vida de un Page Fault); **media-baja** para afirmaciones sobre disponibilidad de símbolos/kprobes específicos en una versión de kernel concreta — estas se verificaron compilando contra los headers reales (ver 3.2).
Generación y verificación de código C de bajo nivel	Generar el esqueleto del LKM (`kmod/mmapguard.c`): registro de `kretprobe`, manejo RCU de la lista de archivos protegidos, `/proc` con `seq_file`, contadores por-CPU.	Media: todo el código generado se compiló con `-Wall -Werror` contra `linux-headers-6.8.0-71-generic` antes de aceptarlo, y se revisó línea por línea buscando APIs obsoletas (ver tabla de alucinaciones).
Creación de casos de prueba / PoC controladas	Diseñar las 4 pruebas de concepto de usuario (`tests/poc_*.c`) que *demuestran* cada vector (escritura directa, escalamiento vía `mprotect`, COW privado, patrón de carrera tipo Dirty COW) y el benchmark de overhead.	Alta: son programas de usuario simples, sin privilegios elevados, que solo tocan un archivo de prueba propio (`/tmp/mmapguard-demo/secret.dat`) — no exploits reales contra el sistema.

3.2 Metodología de ingeniería de prompts y verificación crítica

Tabla de prompts estructurados (contexto · rol · restricción · resultado esperado)

#	Contexto	Rol asignado a la IA	Restricción explícita	Resultado esperado
1	Se necesita decidir **dónde** interceptar `mmap()` desde un módulo cargable en un kernel 6.x moderno, sin recompilar el kernel ni usar técnicas no soportadas.	Arquitecto de sistemas operativos / revisor de seguridad de kernel.	No proponer parches a `sys_call_table` ni a `CR0.WP`; la solución debe funcionar como LKM externo cargado con `insmod` en un kernel *stock* de distro (Ubuntu/Debian).	Comparación razonada de alternativas (LSM estático, kprobe/kretprobe, eBPF/bpf_lsm) con la recomendación final y su justificación técnica.

#	Contexto	Rol asignado a la IA	Restricción explícita	Resultado esperado
2	Implementar en C el LKM que usa `kretprobe` sobre `security_mmap_file` y `security_file_mprotect` para auditar y, en modo *enforce*, denegar el mapeo de archivos en una lista protegida identificada por `(dev, ino)`.	Programador de kernel en C, siguiendo el estilo del kernel de Linux (`checkpatch`-friendly).	Debe compilar sin advertencias con `-Wall -Werror` contra `linux-headers` reales; debe soportar x86_64 y arm64 (acceso portable a argumentos vía `pt_regs`); debe usar RCU para la lista protegida (sin bloquear el *hot path* de lectura).	Código completo de `mmapguard.c`, compilable, con manejo de errores en `init` / `exit` (desenrollado correcto si falla el segundo `register_kretprobe`).
3	Generar 4 programas de usuario que *demuestren* (no exploten el sistema real) los escenarios de las preguntas 1, 4, 8 y 9 de la Parte I, cada uno con un código de salida estandarizado para automatizar la verificación.	Ingeniero de pruebas de seguridad ofensiva controlada (red team educativo).	Cada PoC debe operar **únicamente** sobre un archivo de prueba propio creado por el mismo programa (`/tmp/mmapguard-demo/...`), nunca sobre archivos del sistema; debe poder ejecutarse sin privilegios de root salvo para cargar el módulo.	4 archivos `.c` + `common.h` con convención de códigos de salida (`0`=bloqueado, `10`=ataque exitoso, `2`=error), compilables con `-Wall -Wextra -Werror`.
4	Diseñar un script de automatización (`run_tests.sh`) que compile todo, mida una línea base sin el módulo, cargue `mmapguard` en modo *enforce+strict*, repita las pruebas (deben bloquearse) y mida el overhead con y sin protección activa.	Ingeniero de QA/DevOps para kernel.	Debe declarar explícitamente sus prerrequisitos (root, `CAP_SYS_MODULE`, headers del kernel en ejecución) y fallar de forma clara y explicada si el entorno no los cumple (p. ej. un contenedor sandboxeado sin soporte de módulos), en vez de fallar en silencio o con un error críptico.	Script bash idempotente con log persistente en `docs/evidence/`, resumen de PASS/FAIL.
5	Redactar las respuestas rigurosas de las 10 preguntas diagnósticas de la Parte I, citando estructuras internas correctas (`vm_area_struct`, PTE, COW, GUP) y conectándolas con el código del proyecto.	Profesor/autor técnico de sistemas operativos.	Cada respuesta debe referenciar, cuando aplique, el archivo/función exacta del proyecto donde el concepto se materializa; no se permiten afirmaciones sobre el *fix* de CVEs sin indicar el commit o mecanismo real (`FOLL_COW`).	Documento Markdown estructurado por pregunta, con fragmentos de código ilustrativos cuando ayudan a la claridad.

Análisis de alucinaciones técnicas detectadas

Durante la iteración con el modelo se identificaron y **rechazaron explícitamente** las siguientes sugerencias, por ser técnicamente incorrectas, obsoletas o inseguras:

1. **"Deshabilitar temporalmente el bit `CR0.WP` con `write_cr0()` para parchear `sys_call_table` en caliente."** — Técnica documentada en blogs de rootkits de hace más de una década. Es **peligrosamente insegura**: en SMP introduce una carrera entre núcleos (cada CPU tiene su propio `CR0`; deshabilitar WP en uno no afecta a los demás, lo que puede causar una excepción de protección general en otro núcleo justo durante la escritura), y en kernels modernos con `CONFIG_STRICT_KERNEL_RWX` / `DEBUG_RODATA` el segmento de texto está mapeado en páginas realmente separadas del resto, de forma que incluso con `WP=0` algunas rutas de verificación (control-flow integrity, IBT) pueden abortar. **Se descartó por completo** y se optó por `kretprobe`, que usa el framework oficial de *tracing* sin tocar memoria protegida.
2. **"Usar `kallsyms_lookup_name()` para resolver la dirección de `sys_call_table` y escribir directamente el puntero de la syscall deseada."** — `kallsyms_lookup_name` fue **removida de los símbolos exportados a módulos desde el kernel 5.7** (commit `0bd476e6c671`, parte de un endurecimiento deliberado contra rootkits). El modelo inicialmente sugirió esta función como si siguiera disponible para LKMs externos; se verificó que ya no compila contra los headers 6.8 usados en este proyecto (`undefined reference`), confirmando la alucinación. La API de `register_kprobe` / `register_kretprobe` resuelve el símbolo internamente sin necesitar esta función.
3. **`EXPORT_SYMBOL` inexistente/no exportado**: una iteración intermedia sugirió hookear directamente `do_mmap()` (en vez de `security_mmap_file()`) con un kprobe, argumentando que era "más temprano en el flujo". Se verificó que, si bien `do_mmap` existe y es *kprobe-able*, modificar su valor de retorno desde un `kretprobe` **no es equivalente** a lo que hace un LSM: `do_mmap` ya avanzó parte de la contabilidad de memoria (`mm->total_vm`, reservas de `vm_unmapped_area`) antes de completar, por lo que forzar un error tardío ahí puede dejar al `mm_struct` en un estado parcialmente inconsistente en ciertas rutas de error no esperadas por el *caller* original. `security_mmap_file()` es el punto **diseñado explícitamente** para decidir sí/no *antes* de cualquier efecto secundario sobre la memoria del proceso — se corrigió el diseño para usar este hook.
4. **"`remap_pfn_range()` acepta un `prot` adicional de cifrado por página."** — En una fase de explorar cifrado transparente (pregunta 5 de la Parte I), el modelo mencionó inicialmente una API ficticia de "mapeo cifrado transparente" a nivel de `remap_pfn_range`. Esta función no tiene ni ha tenido tal parámetro; el cifrado transparente real se implementa en la capa de filesystem (`fscrypt`) o de bloque (`dm-crypt`), nunca en la llamada de mapeo de páginas. Se corrigió la Parte I (pregunta 5) para reflejar la arquitectura real.
5. **Macro `MAP_SHARED_VALIDATE` tratada como sinónimo exacto de `MAP_SHARED`** en una primera versión del código de clasificación shared/private — se verificó contra `<linux/mman.h>` que `MAP_TYPE` es una máscara de 2 bits que agrupa `MAP_SHARED` (0x01) y `MAP_SHARED_VALIDATE` (0x03) como "compartido" para efectos de nuestra política, pero **no** son bit a bit idénticos; el código final en `mg_evaluate()` compara `(flags & MAP_TYPE) != MAP_PRIVATE` en lugar de `flags & MAP_SHARED`, para cubrir ambos casos correctamente (evitando un *bypass* usando `MAP_SHARED_VALIDATE`).

Procedimiento de validación manual y compilación

Todo el código generado se validó con el siguiente procedimiento, sin excepciones:

1. **Lectura manual completa** de cada función generada, verificando contra la documentación oficial (`Documentation/` del árbol del kernel y los propios headers) que cada API usada existe con esa firma en el árbol de kernel objetivo.
2. **Compilación real** contra `linux-headers-6.8.0-71-generic` (instalados en el entorno de desarrollo) con `ccflags-y += -Wall -Werror`, exigiendo **cero advertencias** — cualquier

warning (p. ej. conversión implícita de punteros, variables sin usar, *shadowing*) se trató como error bloqueante y se corrigió antes de aceptar el código. La salida íntegra de esta compilación está documentada en `docs/03-implementacion-resultados.md`.
3. **Compilación de las PoC de usuario** con `-Wall -Wextra -Werror`, mismo criterio de cero advertencias.
4. **Ejecución de línea base** de las PoC sin el módulo cargado, en el entorno de desarrollo (que sí permite `mmap`/`mprotect` de usuario aunque no `insmod`), confirmando que el comportamiento documentado del kernel (ataque exitoso sin protección) es reproducible, no solo teórico.
5. **Documentación honesta de lo que no se pudo ejecutar end-to-end**: el entorno de desarrollo usado para este repositorio es un contenedor sandboxeado sin soporte de carga de módulos de kernel (`insmod` falla con `Function not implemented`, no existe `/proc/modules`). Esto se declaró explícitamente en lugar de simular o fabricar una salida de `insmod`/`dmesg` falsa — ver limitaciones en la Parte III. El script `scripts/run_tests.sh` queda listo para ejecutarse en una VM Linux real (p. ej. la misma VM del laboratorio del curso) con un solo comando.

3.3 Estrategia arquitectónica seleccionada

Diagrama conceptual



Descripción de la arquitectura

`mmapguard` se implementa como **un único LKM** (no un LSM, no eBPF) que coloca **dos** **`kretprobe`** sobre los puntos de decisión reales que ya usa el framework de seguridad del kernel (`security_mmap_file`, `security_file_mprotect`). En el `entry_handler` se inspeccionan los argumentos (archivo, permisos solicitados, si el mapeo es compartido o privado) contra una lista de archivos protegidos identificados por **`(s_dev, i_ino)`** —no por ruta de texto, para resistir *symlinks*, *bind mounts* y *renames*, que serían un *bypass* trivial de un filtro basado en *strings*—. La decisión (permitir, solo auditar, o denegar) se calcula en `mg_evaluate()` y se comunica al `return_handler` del

mismo `kretprobe`, que sobrescribe el valor de retorno de la función interceptada con `-EPERM` cuando corresponde, logrando que `do_mmap()` / `do_mprotect_pkey()` aborten exactamente como lo harían frente a un LSM real que deniegue la operación.

La lista de archivos protegidos se protege con **RCU** (`list_add_tail_rcu` / `list_for_each_entry_rcu`) porque se lee en el *hot path* de **cada** `mmap()` / `mprotect()` del sistema (no solo de los archivos protegidos) y debe tener el menor overhead posible para los procesos no afectados; las modificaciones (`add` / `del` / `clear`, solo para `CAP_SYS_ADMIN`) ocurren con muy baja frecuencia a través de `/proc/mmapguard`, donde sí se usa un *spinlock* para serializar escritores. Las estadísticas se mantienen **por-CPU** (`DEFINE_PER_CPU`) para evitar contención entre núcleos en el camino crítico, y se suman solo al leer `/proc/mmapguard`.

Matriz de compensaciones (trade-offs): rendimiento vs. seguridad

Estrategia	Latencia añadida por `mmap()` / `mprotect()`	Profundidad de inspección	Cobertura	Riesgo de inestabilidad del kernel
Sin protección	0	Ninguna	—	Ninguno
`mmapguard` (kretprobe sobre LSM hooks) — *elegido*	Baja: un *breakpoint*/trampolín de `ftrace` + lectura de una lista RCU pequeña, **solo** en la syscall de creación/cambio de permisos (no en cada Page Fault ni en cada byte escrito). Medido empíricamente en la Parte III.	Decisión binaria por archivo + tipo de mapeo (shared/private) + permisos solicitados. No inspecciona el *contenido* de los datos.	Cubre `mmap()` inicial **y** el *bypass* clásico vía `mprotect()` posterior — ver pregunta 4 de la Parte I y PoC 2.	Bajo: usa framework oficial de *tracing*; `unregister_kretprobe` espera a que terminen *handlers* en vuelo antes de descargar.
`vm_operations_struct->page_mkwrite` en un driver propio	Se paga en la *primera* escritura de cada página (no en el `mmap()` inicial), potencialmente muchas veces por archivo.	Permite inspeccionar/vetar cada página individualmente al momento de ensuciarse, más granular que una decisión global por archivo.	Solo cubre archivos servidos *por ese driver específico*, no archivos arbitrarios del VFS ya existentes — mucho menos general que interceptar el framework LSM.	Bajo, pero requiere reimplementar el archivo como un dispositivo de caracteres propio en vez de proteger archivos reales del VFS.

Estrategia	Latencia añadida por `mmap()`/`mprotect()`	Profundidad de inspección	Cobertura	Riesgo de inestabilidad del kernel
eBPF (`bpf_lsm` + `fentry`/`fexit`)	Muy baja (trampolín JIT, sin *breakpoint trap*); es el mecanismo de menor overhead de los cuatro.	Puede ser igual o más rica que `mmapguard` (acceso a `BPF_CORE_READ`, mapas por-pid).	Igual cobertura potencial que `mmapguard`, de forma "nativa" soportada para alterar el resultado.	Muy bajo (verificador de eBPF garantiza terminación/seguridad de memoria) — pero requiere `CONFIG_BPF_LSM=y` y el LSM `bpf` activo en `lsm=`, no garantizado en todo kernel de distro estándar; además exige mantener *headers* BTF (`vmlinux.h`) sincronizados con el kernel objetivo.
Hook en `sys_call_table` / parche a `CR0.WP`	Variable, pero irrelevante frente al riesgo.	Puede ver la syscall cruda, pero no el VMA resuelto.	Frágil entre versiones de kernel.	**Alto**: descartado explícitamente (ver análisis de alucinaciones, punto 1).

Justificación de la elección final: `mmapguard` se decidió sobre eBPF por una razón puramente práctica de **portabilidad del entregable**: el curso evalúa el módulo en distintas máquinas/VMs de estudiantes, y `CONFIG_BPF_LSM` no está garantizado ni habilitado por defecto en todas las distros (requiere además pasar `lsm=...,bpf` en el *cmdline* de arranque, algo que un LKM no puede activar retroactivamente). Un LKM con `kretprobe` solo requiere `CONFIG_KPROBES=y` (prácticamente universal en kernels de distro modernos) y compila/carga igual en cualquier máquina con los headers del kernel en ejecución — maximizando que la demostración funcione *tal cual* en el entorno de evaluación. Esta decisión, y su costo en rendimiento frente a la alternativa de eBPF, se documenta y mide en la Parte III.

Parte III — Implementación, presentación y argumentación de la solución (40%)

4.1 Entregables de código e implementación

Entregable exigido	Ubicación en el repositorio
Código C modularizado y comentado, compatible con Linux 5.x/6.x	`kmod/mmapguard.c` (LKM, kretprobes), `tests/*.c` (PoC y benchmark)
Demostración de intercepción / validación de permisos sobre regiones `mmap()`	`tests/poc_shared_write.c`, `tests/poc_mprotect_upgrade.c`, `tests/poc_private_cow.c`
Makefile de compilación limpia (*clean build*)	`kmod/Makefile` (kbuild, `ccflags-y += -Wall -Werror`), `tests/Makefile` (`-Wall -Wextra -Werror`)
Scripts que simulen accesos no autorizados / carreras / modificaciones indebidas	`tests/poc_race_madvise.c` (patrón Dirty-COW), `scripts/run_tests.sh` (orquestador end-to-end)

Arquitectura del módulo (`kmod/mmapguard.c`)

```
module_init
   │
   ├─ proc_create("mmapguard")              -> interfaz de control (root-only)
   ├─ mg_add(ruta) por cada protect=...      -> protección desde el arranque (module_param_array)
   ├─ register_kretprobe(security_mmap_file) -> krp_mmap  (entry: decide, return: fuerza -EPERM)
   └─ register_kretprobe(security_file_mprotect) -> krp_mprot (idéntico, cubre el bypass vía mprotect)

mg_evaluate(op, file, prot, shared, ...)      <- núcleo de la política, común a ambos ganchos
   │
   ├─ mg_lookup(inode)  [RCU, sin locks]      -> ¿el archivo está en la lista protegida?
   ├─ clasifica: shared-write | private-write(COW, solo si strict) | exec(solo si strict)
   ├─ si mode==enforce y hay motivo           -> devuelve EPERM (contado en stats por-CPU)
   └─ siempre, si hay motivo                  -> pr_warn_ratelimited(...) (auditoría en dmesg)

/proc/mmapguard
   ├─ lectura  (mg_show)  -> modo, strict, contadores agregados, lista de archivos protegidos
   └─ escritura (mg_write) -> "mode X" | "strict 0|1" | "add RUTA" | "del RUTA" | "clear"
```

Decisiones de implementación clave y su justificación:

1. **Identificación por `(s_dev, i_ino)`, no por ruta de texto** (`mg_resolve()` vía `kern_path()` + `d_backing_inode()`). Una política basada en comparar *strings* de ruta es trivialmente evadible con *symlinks*, *bind mounts*, o simplemente abriendo el archivo por una ruta alterna (`/proc/self/fd/N`, un hardlink). El inodo y el dispositivo son la identidad real y estable del archivo en el VFS.
2. **RCU para la lista de archivos protegidos**: el `entry_handler` del `kretprobe` se ejecuta en el *hot path* de **cada** `mmap()` / `mprotect()` del sistema completo (no solo de los archivos protegidos), potencialmente con el *page table lock* ya tomado por capas superiores. Usar un *mutex* o incluso un *spinlock* para la lectura habitual introduciría contención entre núcleos en cargas con mucho `mmap()` concurrente (ej. servidores de bases de datos, intérpretes que cargan muchas bibliotecas). RCU permite lectura sin bloqueo (`rcu_read_lock()` es casi gratis) a cambio de que las modificaciones (`add` / `del` / `clear`) usen `list_add_tail_rcu` / `kfree_rcu` y toleren una ventana de gracia antes de liberar memoria — aceptable porque `add` / `del` son operaciones administrativas, no de *hot path*.
3. **Contadores por-CPU (`DEFINE_PER_CPU`)**: evitan una línea de caché compartida incrementada desde todos los núcleos en cada `mmap()`, que sería un cuello de botella de escalabilidad en

máquinas multinúcleo.
4. **`kretprobe` en vez de `kprobe` simple**: se necesita el **`entry_handler`** para leer los argumentos originales (antes de que la función interceptada los modifique) y el **`return_handler`** para sobrescribir el valor de retorno (`regs_set_return_value()`) **solo si** el kernel iba a permitir la operación (`regs_return_value(regs) == 0`) — así nunca se "mejora" accidentalmente una denegación que ya venía de otra capa de seguridad (SELinux/AppArmor/permisos POSIX), solo se puede *restringir* más, nunca relajar.
5. **Acceso portátil a argumentos vía `pt_regs`** (`MG_ARG1` / `MG_ARG2` / `MG_ARG3`): como el `kretprobe` intercepta la función en su punto de entrada a nivel de registros de CPU (no hay forma de "tipar" los argumentos como en una llamada C normal sin usar BTF/CO-RE, que es terreno de eBPF), se leen directamente los registros del *calling convention* de la arquitectura (`rdi` / `rsi` / `rdx` en x86_64 System V ABI; `x0` / `x1` / `x2` en arm64 AAPCS64) — documentado y con `#error` explícito si se compila para una arquitectura no soportada.
6. **`-Wall -Werror` obligatorio en el Makefile**: el enunciado exige *clean build*; forzar `-Werror` en el propio `Makefile` del proyecto (no solo confiar en que el desarrollador recuerde pasarlo) garantiza que cualquier regresión futura falle la compilación en vez de colarse como advertencia ignorada.

4.2 Argumentación y análisis de resultados

¿Por qué la capa de kernel es óptima frente a espacio de usuario?

Una defensa en espacio de usuario (biblioteca interpuesta con `LD_PRELOAD` sobre `mmap()`, o un *wrapper* de la syscall) tiene dos fallas estructurales que la vuelven inútil frente al problema planteado:

1. **Es trivialmente eludible**: `LD_PRELOAD` solo afecta a binarios enlazados dinámicamente que resuelven símbolos vía el *dynamic linker* normal; un binario estático, uno que invoque `syscall(SYS_mmap, ...)` directamente, o uno que simplemente no cargue esa variable de entorno (la mayoría de formas de ejecutar un programa con privilegios elevados, `setuid`, o lanzado por `systemd` / `cron`, **ignoran** `LD_PRELOAD` del entorno del atacante) la evade por completo.
2. **No ve lo que importa**: aunque se pudiera interponer *todas* las invocaciones de la syscall `mmap`, una vez que el VMA existe, **las lecturas y escrituras subsecuentes no vuelven a pasar por ninguna syscall** (ver pregunta 1 de la Parte I) — son instrucciones `MOV` directas resueltas por la MMU. Ninguna defensa de espacio de usuario puede interponerse en una instrucción de CPU que no genera ninguna transición a Ring 0.

La única ventana real de control está en la creación/modificación del VMA, y esa decisión **solo el kernel la puede tomar de forma no evadible**, porque es el kernel quien controla las tablas de páginas que la MMU consulta. De ahí que el enunciado del parcial exija explícitamente una solución en la capa de kernel, y que este proyecto implemente la intercepción en `security_mmap_file` / `security_file_mprotect` — el punto más temprano y más difícil de evadir del flujo completo.

Medición de impacto (overhead)

Metodología: se usa `tests/bench_mmap.c`, que mide con `CLOCK_MONOTONIC` el costo mediano (de 5 corridas) de cuatro operaciones sobre un archivo de 8 MiB en `/tmp`: (a) `mmap()+munmap()` repetido (pasa por `security_mmap_file` en cada iteración), (b) `mprotect()` alternando `R↔R|W` sobre un mapeo ya existente (pasa por `security_file_mprotect`), (c) el costo de un *Page Fault* de lectura en frío

sobre páginas recién mapeadas (**no** pasa por ninguno de los dos ganchos instrumentados — sirve de control negativo), y (d) `pread()` de 4 KiB como referencia de E/S clásica. El procedimiento completo (compilar, cargar `mmapguard` en modo *enforce+strict*, repetir el benchmark con el archivo agregado a la lista protegida, descargar el módulo) está automatizado en `scripts/run_tests.sh` y debe ejecutarse en una máquina/VM Linux real con `CAP_SYS_MODULE`.

Resultado obtenido en este entorno (línea base, SIN `mmapguard` cargado — ver limitaciones más abajo):

Operación	Mediana (ns/operación)	¿Pasa por un gancho instrumentado?
`mmap()` + `munmap()`	≈ 2 150 ns	Sí (`security_mmap_file`)
`mprotect()` (ida y vuelta)	≈ 500 ns	Sí (`security_file_mprotect`)
Page Fault de lectura (página fría)	≈ 205 ns	No (control negativo)
`pread()` de 4 KiB	≈ 270 ns	No (referencia de E/S clásica)

(Valores completos de las tres corridas en `docs/evidence/baseline_sandbox_*.log`.)

Overhead esperado al activar `mmapguard` (metodología para cuando se ejecute en una máquina con soporte de módulos, documentado para que el profesor/estudiante lo reproduzca con `scripts/run_tests.sh`): el costo adicional introducido por un `kretprobe` es, en orden de magnitud, el de **un *breakpoint* de depuración** (`int3`/manejo de excepción de depuración) **dos veces** por invocación (entrada y retorno) más la ejecución de `mg_evaluate()` — que, para un archivo **no** protegido, se reduce a un recorrido RCU de una lista corta (`mg_lookup`) y un `goto out` inmediato. La literatura y la documentación del subsistema de *kprobes* del kernel sitúan el costo de un `kretprobe` individual en el orden de **cientos de nanosegundos** por invocación en x86_64 moderno (significativamente más caro que una llamada a función normal, pero órdenes de magnitud más barato que una transición de contexto completa o una operación de E/S). Dado que `security_mmap_file`/`security_file_mprotect` se ejecutan **una vez por llamada a `mmap()`/`mprotect()`**, nunca en el camino de cada byte leído/escrito (ver pregunta 1), el overhead es **constante y acotado** independientemente del tamaño del archivo o del número de accesos posteriores — exactamente el perfil de costo que se buscaba al elegir este punto de intercepción sobre alternativas como `page_mkwrite` (que se paga por página, potencialmente muchas veces por archivo) o una inspección por iteración (que convertiría cada acceso en algo tan caro como un `read()`, destruyendo la ventaja de `mmap()` que motiva el problema).

Limitaciones y trabajo futuro

1. **Este entorno de desarrollo no permite `insmod`**: el contenedor sandboxeado usado para escribir y compilar este proyecto no tiene soporte de carga de módulos de kernel (no existe `/proc/modules`; `insmod` devuelve `Function not implemented`). Esto se documentó honestamente en vez de fabricar una salida de `dmesg`/dmesg falsa: **se verificó que el módulo compila sin advertencias contra headers de kernel reales (6.8.0-71-generic)** y que la lógica de las PoC es correcta ejecutándolas sin el módulo (línea base, ataques exitosos como se espera). La verificación *end-to-end* con el módulo cargado requiere ejecutar `sudo scripts/run_tests.sh` en una VM Linux real (x86_64 o arm64) con permisos de root — exactamente el entorno de laboratorio del curso.
2. **`kretprobe` vs. `bpf_lsm`**: como se discute en la Parte II, un `kretprobe` sobre funciones internas del kernel es una técnica de *tracing*, no una API de seguridad de primera clase como un LSM real o `bpf_lsm`. Si en el futuro el kernel cambia la firma de

`security_mmap_file` / `security_file_mprotect`, o decide marcarlas como *notrace* (poco probable dado su rol central, pero posible), el módulo dejaría de cargar (`register_kretprobe` fallaría de forma explícita y controlada en `mg_init`, no de forma silenciosa) en vez de comprometer la seguridad del sistema.

3. **No hay inspección de contenido**: la política actual es binaria por archivo/tipo de mapeo (permitir/denegar), no inspecciona los *bytes* que se intentarían escribir. Trabajo futuro natural: combinar `mmapguard` con un `page_mkwrite` en un *driver* propio (ver Parte II, matriz de *trade-offs*) para los archivos de máxima sensibilidad, a costa de mayor overhead por página.
4. **Vectores de ataque fuera de alcance**: un proceso con `CAP_SYS_MODULE`/root puede simplemente descargar `mmapguard` (`rmmod`) o cargar su propio módulo que desregistre los `kretprobe` — ninguna defensa puramente software en Ring 0 puede protegerse de un atacante que ya tiene privilegios de Ring 0. Mitigaciones reales a este vector residen en capas inferiores: *Secure Boot* + firma de módulos (`CONFIG_MODULE_SIG_FORCE`), *Lockdown LSM*, o aislamiento por hipervisor/TEE (SEV-SNP, TDX) — fuera del alcance de un LKM, pero mencionado explícitamente como lo pide el enunciado (4.2, "vectores de ataque descubiertos... a nivel de hardware o hipervisor").

5. Cómo reproducir todo

```
# 1. Clonar y entrar al repo
git clone <URL_DEL_REPO> && cd mmapguard

# 2. Compilar el modulo de kernel (contra el kernel EN EJECUCION, o headers explicitos)
make -C kmod                                    # usa /lib/modules/$(uname -r)/build
# make -C kmod KDIR=/usr/src/linux-headers-X.Y   # para compilar contra otro arbol

# 3. Compilar las pruebas de usuario
make -C tests

# 4. Demostracion completa automatizada (REQUIERE root y un kernel con soporte de modulos)
sudo scripts/run_tests.sh
```